ASSIGNMENT 4 – CO4

DESIGNING REUSABLE CLASS HIERARCHIES

Course: Object Oriented Programming with C++ (DSA01)

Course Outcome: CO4 – Analyse and implement C++ applications using inheritance, types of inheritance, virtual base classes, abstract classes, pointers and polymorphism to design reusable and extensible programs (constructors in derived classes, member classes, pointers to objects, the this pointer and pointers to derived classes)

NAME: JEEVANESH WARAN.S REG NO : 192421331

Nature of Assignment: Individual

Duration: One Week

Total Marks: 50

Submission Date: 11-11-2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment4 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192421331/DSA0103-C-

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

PURPOSE OF THE ASSIGNMENT

This assignment evaluates your ability to design a class hierarchy before writing its code, so that common behaviour is written once and new kinds of objects can be added without changing working classes.

You are expected to demonstrate the following progression:

Interpret the Requirement → Identify Is-a and Has-a Relationships → Design the Hierarchy → Decide Access → Implement → Trace Construction → Challenge → Refine

Where pseudocode is required, it must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode. All code must compile with g++ -std=c++17 -Wall -Wextra.

TASK 1 – CLASS DESIGN REVIEW

Bloom's Level: K5 – Evaluate

Diagnose and Repair Duplicated Classes

6 Marks

A college payroll program has three separate classes written by different developers:

```
class Professor   { public: string name; int id; double basic; double da; double research;
double pay() { return basic + basic * 0.42 + research; } }; class LabAssistant{ public:
string name; int id; double basic; double da;                double pay() { return basic +
basic * 0.42; } }; class GuestFaculty{ public: string nm;   int code; double hours;
double rate;                 double pay() { return hours * rate; } };
```

Task 1.1 – Diagnose the Design | 2 Marks

Identify at least five design deficiencies. Examine aspects such as:

- Repeated fields and repeated pay rules
- Public data members
- Inconsistent names for the same information
- The unused da field
- The magic number 0.42
- The effort needed to add a new staff category
- The inability to process all staff in one loop

ANSWER: Diagnose the Design

The given payroll classes have the following design deficiencies:

1. **Code duplication:** Professor and LabAssistant contain repeated data members such as name, id and basic.
2. **Public data members:** Public variables allow direct modification and reduce data security.
3. **Inconsistent naming:** Professor uses name and id, whereas GuestFaculty uses nm and code.
4. **Unused variable:** The da member is declared but never used in the pay calculation.
5. **Magic number:** The value 0.42 is directly used instead of defining a meaningful constant.
6. **Poor extensibility:** Adding a new staff category requires modifying multiple parts of the program.
7. **No polymorphism:** All staff types cannot be processed conveniently using one common base-class pointer.

Task 1.2 – Repair the Design | 3 Marks

Redesign the classes as a hierarchy with a common base class, protected or private data, and a pay() function overridden in each derived class. Draw the UML class diagram.

ANSWER: Repair the Design

Improved C++ program:

main.cpp

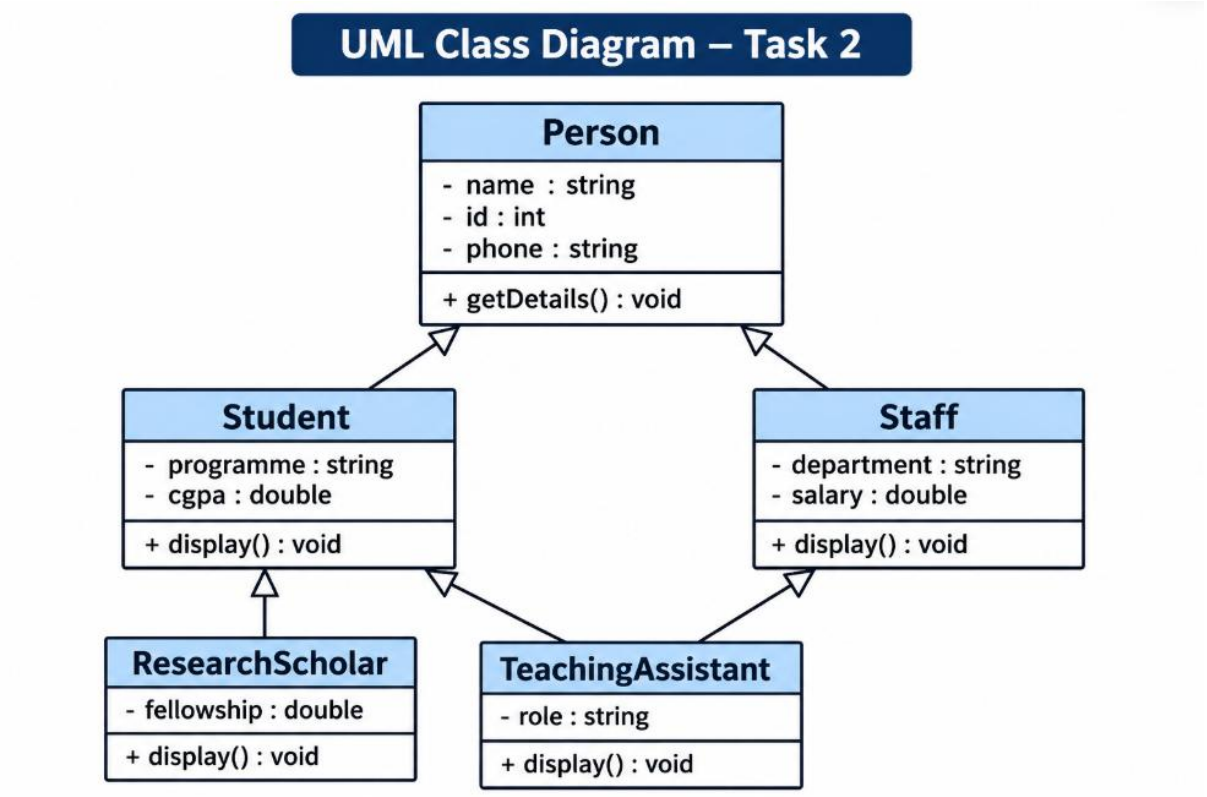
Clear Run ▶

Output

```
1
2  #include <iostream>
3  #include <string>
4  using namespace std;
5
6  const double DA_RATE = 0.42;
7
```

```
Arun 101 61800
Kumar 102 35500
Ravi 103 20000
```

UML Class Diagram:



Task 1.3 – Design Defence | 1 Mark

In not more than 100 words, explain how your hierarchy makes adding a new staff category easier.

Alternative Concept and Complexity Analysis

If the three staff types were represented by one Staff class with a type field and a switch inside pay(), instead of a base class with derived classes, what would the solution be? Analyse the time per pay() call (O(1) in both) and the space per object (unused fields in the single class), and count the places that must change when a new staff type is added.

Evidence Expected

A reasoned diagnosis, a corrected hierarchy with a UML diagram and a concise justification.

ANSWER: Design Defence

The class hierarchy improves reusability by storing common staff information in a single base class. Each derived class implements its own pay() function using function overriding. Polymorphism allows all staff members to be processed through a common base-class pointer. When a new staff category is introduced, a new derived class can be created without modifying the existing classes.

Alternative Concept and Complexity Analysis

If a single Staff class with a type variable and switch statement is used, all staff details must be stored in one class.

- Time complexity of pay() = O(1).
- Space complexity = O(1) per object, but unused fields consume memory.
- Adding a new staff type requires changing the type definition and switch statement.
- In inheritance, a new derived class can be added independently.

Conclusion: Inheritance provides better reusability and extensibility.

TASK 2 – FROM REQUIREMENT TO HIERARCHY

Bloom's Level: K5 – Evaluate

University People Management

8 Marks

A university keeps records of persons (name, ID, phone). Students have a programme and CGPA; staff have a department and salary. A teaching assistant is both a student and a staff member. Research scholars are students who also hold a fellowship.

Task 2.1 – Identify the Relationships | 2 Marks

Classify each relationship as is-a (inheritance) or has-a (member class), and name the type of inheritance (single, multiple, multilevel, hierarchical, hybrid) used for each part.

ANSWER: FROM REQUIREMENT TO HIERARCHY (8 Marks)

2.1 Identify the Relationships

Relationship	**Type**	**Inheritance**
Student inherits Person	Is-a	Single
Staff inherits Person	Is-a	Single
TeachingAssistant inherits Student and Staff	Is-a	Multiple
ResearchScholar inherits Student	Is-a	Multilevel

Student and Staff share Person	Is-a	Hierarchical
Teaching Assistant structure	Is-a	Hybrid

Address and Course can be represented as member classes using composition (has-a).

Task 2.2 – Resolve the Diamond | 2 Marks

Explain the problem that arises when TeachingAssistant inherits from both Student and Staff, and how virtual base classes solve it.

ANSWER: Resolve the Diamond

The diamond problem occurs when TeachingAssistant inherits from both Student and Staff, and both classes inherit from Person.

Without virtual inheritance, TeachingAssistant contains two separate Person subobjects. This creates ambiguity when accessing common members such as name and ID.

Solution:

Use virtual base classes.

main.cpp

```
class Student : virtual public Person {
};

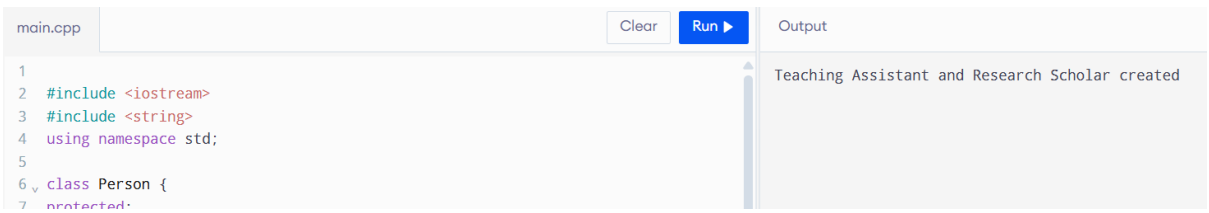
class Staff : virtual public Person {
};

class TeachingAssistant : public Student, public Staff {
};
```

Task 2.3 – Design the Classes | 3 Marks

Write the class declarations with access specifiers, and decide for each base class whether it is inherited publicly, protectedly or privately.

ANSWER: Design the Classes



Task 2.4 – Access Table | 1 Mark

Prepare a table showing how a public, protected and private member of Person is visible in Student under public, protected and private inheritance.

ANSWER: Access Table

Assume Person has public, protected and private members.

Person member	**Public inheritance**	**Protected inheritance**	**Private inheritance**
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not directly accessible	Not directly accessible	Not directly accessible

Predict the Output : **Person Student Staff TA**

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output. #include <iostream>

```
struct Person  { Person()  { std::cout << "Person "; } }; struct Student :
virtual Person { Student() { std::cout << "Student "; } }; struct Staff   :
virtual Person { Staff()   { std::cout << "Staff "; } }; struct
TeachingAssistant : Student, Staff {
   TeachingAssistant() { std::cout << "TA "; }
};

int main()
{
   TeachingAssistant ta;
std::cout << '\n';
}
```

Explanation:

1. Person constructor executes first because it is the virtual base class.
2. Student constructor executes next.

3. Staff constructor executes next.
4. Teaching Assistant constructor executes last.

Person is printed only once because virtual inheritance shares one Person subobject.

After removing virtual:

struct Student : Person {};

struct Staff : Person {};

My predicted output (write before running): **Person Student Person Staff TA**

Explain why Person is printed only once. Remove the word virtual from both base classes, predict the new output and compiler messages, and verify.

Alternative Concept and Complexity Analysis

If TeachingAssistant contained a Student object and a Staff object as members (composition, has-a), instead of inheriting from both through virtual base classes, what would the solution be? Analyse the space per object (one or two Person sub-objects) with sizeof, and compare the time and ambiguity of reaching the name field in each design.

Evidence Expected

A relationship analysis, a resolved diamond, class declarations and a correct access table.

ANSWER: Alternative Concept and Complexity Analysis

If TeachingAssistant contains Student and Staff as member objects, it uses composition instead of multiple inheritance.

- It contains separate Student and Staff objects.
- Each object contains its own Person subobject.
- Therefore, there are two Person subobjects.
- Access to members is through student.name or staff.name.
- This avoids inheritance ambiguity but does not provide the same is-a relationship.

Conclusion: Virtual inheritance is suitable when TeachingAssistant must be both Student and Staff while sharing one Person object.

TASK 3 – CONSTRUCTORS IN DERIVED CLASSES AND MEMBER CLASSES

Bloom's Level: K5 – Evaluate

Order of Construction and Destruction

9 Marks

Task 3.1 – Pass Values to Base Constructors | 3 Marks

Write constructors for Student, Staff and TeachingAssistant that pass values to their base classes through initializer lists. Explain why the most-derived class must call the virtual base constructor directly.

ANSWER: Pass Values to Base Constructors

Constructors in derived classes use member initializer lists to pass values to base-class constructors.

Example:

Student(string n, int i, string p, string pr, double c)

: Person(n, i, p), programme(pr), cgpa(c) {}

TeachingAssistant must directly initialize the virtual base class Person because the most-derived class is responsible for constructing virtual base classes.

Task 3.2 – Member Classes | 2 Marks

Add an Address member class to Person and a Course member class to Student. Show how their constructors receive values.

ANSWER: Member Classes

Address is a member class of Person, and Course is a member class of Student.

```
</> C++

class Address {
    string city;
    int pin;

public:
    Address(string c, int p) : city(c), pin(p) {}
};

class Course {
    string courseName;
    int credits;

public:
    Course(string c, int cr)
        : courseName(c), credits(cr) {}
};
```

Example of using member classes:

```
class Person {
protected:
    string name;
    Address address;

public:
    Person(string n, string c, int p)
        : name(n), address(c, p) {}
};
```

Task 3.3 – Trace the Order | 3 Marks

Print a message in every constructor and destructor and record the order for a Student, a Staff and a TeachingAssistant object. Explain the rule you observe.

ANSWER: Trace the Order

Construction and destruction order:

Object	**Construction order**	**Destruction order**
Student	Person → Address → Course → Student	Student → Course → Address → Person
Staff	Person → Staff	Staff → Person
TeachingAssistant	Person → Student → Staff → TeachingAssistant	TeachingAssistant → Staff → Student → Person

The rules are:

1. Virtual base classes are constructed first.
2. Other base classes are constructed next, in declaration order.
3. Member objects are constructed in their declaration order.
4. Derived-class constructor executes last.
5. Destruction takes place in the reverse order.

Task 3.4 – Experiment | 1 Mark

Remove the virtual keyword from the base classes and record how the output and sizeof(TeachingAssistant) change.

ANSWER: Experiment

When virtual is removed:

- Two separate Person subobjects are created in TeachingAssistant.
- Person constructor executes twice.
- Memory size of TeachingAssistant generally increases.

- Member access may become ambiguous.

The exact sizeof() result depends on the compiler and platform.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

struct Engine {
   Engine() { std::cout << "Engine "; }
   ~Engine() { std::cout << "~Engine "; }
};

struct Vehicle {
   Vehicle() { std::cout << "Vehicle "; }
   ~Vehicle() { std::cout << "~Vehicle "; }
};

struct Car : Vehicle {
   Engine e;
   Car() { std::cout << "Car "; }
   ~Car() { std::cout << "~Car "; }
};

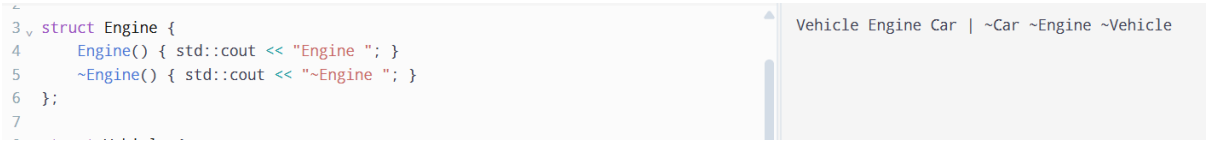
int main()
{
   {        Car c;
std::cout << "| ";
```

} std::cout

<< '\n';

}

My predicted output (write before running): **Vehicle Engine Car | ~Car ~Engine ~Vehicle**



Explanation:

- Vehicle is constructed first because it is the base class.
- Engine is constructed next because it is a member object.
- Car constructor executes last.
- When the object is destroyed, Car destructor executes first.
- Engine destructor executes next.
- Vehicle destructor executes last.

State the rule for the order in which base classes, member objects and the derived class are constructed and destroyed.

Complete the Code

Complete TODO 1 below using a member initializer list, and execute the program.

#include <iostream>

#include <string>

class Person {

protected:

std::string name;

int age;

public:

```
    Person(const std::string& n, int a) : name(n), age(a) {}
};

class Address {                    // used as a member class (composition)
std::string city;     int pin;

public:
    Address(const std::string& c, int p) : city(c), pin(p) {}
std::string str() const { return city + "-" + std::to_string(pin); } };

class Student : public Person {
long long regNo;
    Address home;

public:
    // TODO 1: Write the constructor: initialise the Person base, regNo and the home
//   Address member in the initializer list



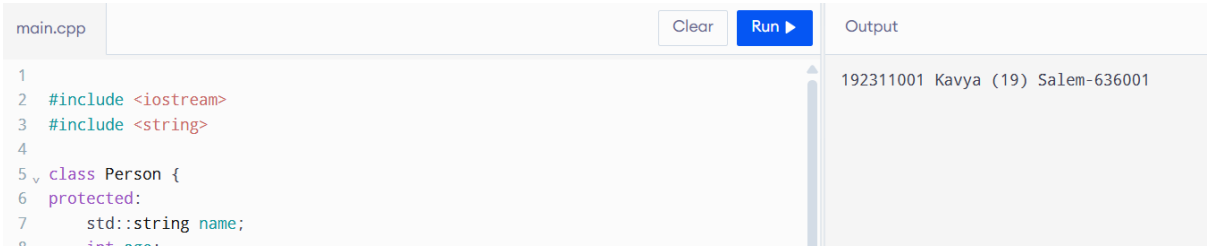
    void print() const
    {
        std::cout << regNo << ' ' << name << " (" << age << ") " << home.str() << '\n';
    }
};

int main()
{
    Student s("Kavya", 19, 192311001, "Salem", 636001);
    s.print();
```

}

Expected output:

192311001 Kavya (19) Salem-636001



Swap the order of the items in your initializer list, compile with -Wall and record the warning.

Alternative Concept and Complexity Analysis

If Student held a pointer to a dynamically created Address (Address* home = new Address(…)) instead of an Address member object, what would the solution be? Analyse the extra time for allocation and the extra space for the pointer, and state what the constructor, copy constructor and destructor must now do.

Evidence Expected

Correct derived-class constructors, member classes, traced construction and destruction orders and an explained experiment.

ANSWER: Alternative Concept and Complexity Analysis

If Address is stored using a pointer:

Address* home = new Address("Salem", 636001);

The following changes are required:

- Constructor must dynamically allocate Address using new.
- Copy constructor must perform a deep copy.
- Destructor must release memory using delete.
- Copy assignment operator must also be implemented safely.

Comparison	**Member object**	**Pointer**
Memory allocation	Automatic	Dynamic
Allocation time	No separate heap allocation	O(1)

Extra storage	Address object	Pointer
Memory management	Automatic	Manual
Memory leak risk	Low	Higher

Conclusion: Composition using a member object is simpler and safer when dynamic allocation is not required.

TASK 4 – POINTERS TO DERIVED CLASSES: FORENSICS

Bloom's Level: K5 – Evaluate

7 Marks

A bank stores different kinds of accounts through base-class pointers. Senior-citizen accounts should earn 4.0% interest, but every account earns 3.5%, and memory reports show that part of each senior account is never released. class Account { public: double rate() const { return 3.5; } ~Account() {} }; class SeniorAccount : public Account { double* history = new double[100]; public: double rate() const { return 4.0; } ~SeniorAccount() { delete[] history; } };

Account* accounts[2] = { new Account, new SeniorAccount };

for (Account* a : accounts) std::cout << a->rate() << "\n"; for

(Account* a : accounts) delete a;

Account copy = SeniorAccount(); // stored in a vector<Account> elsewhere

Task 4.1 – Locate the Defects | 2 Marks

Identify why the senior rate is never used, why memory leaks, and what object slicing does in the last line.

ANSWER: Locate the Defects

1. **Missing virtual function**: rate() is not virtual in Account, so the base-class version is called through an Account pointer.
2. **Missing virtual destructor:** Deleting a SeniorAccount through an Account pointer causes undefined behaviour because the base destructor is not virtual.
3. **Memory leak:** The dynamically allocated history array may not be released correctly.

4. **Object slicing:** Assigning SeniorAccount to Account copies only the Account portion and loses derived-class information.

Task 4.2 – Construct Evidence | 2 Marks

Run the program under -fsanitize=address and add output to the destructors to prove each defect.

ANSWER: Construct Evidence

Compile using:

g++ -std=c++17 -Wall -Wextra -fsanitize=address task4.cpp -o task4

Run:

./task4

Add destructor messages:

```
~Account() {
    cout << "Account destructor called\n";
}


~SeniorAccount() {
    cout << "SeniorAccount destructor called\n";
    delete[] history;
}
```

Expected observations:

- The non-virtual function calls the base-class rate.
- Deletion through a base pointer without a virtual destructor is undefined behaviour.
- AddressSanitizer may report memory errors, but exact diagnostics depend on the compiler and runtime.

Task 4.3 – Repair the Program | 2 Marks

Correct the program using virtual functions, a virtual destructor and pointers (or references) instead of copies.

ANSWER:



Task 4.4 – Prevent Recurrence | 1 Mark

State two rules that would have prevented these defects (for example: a base class with virtual functions needs a virtual destructor; use override).

ANSWER: Prevent Recurrence

1. Always declare a virtual destructor in a polymorphic base class.
2. Use the override keyword when overriding virtual functions.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

#include <iostream>

struct Account { double rate() const {

return 3.5; } virtual double vrate() const {

return 3.5; } virtual ~Account() = default;

};

struct SeniorAccount : Account { double

rate() const { return 4.0; } double vrate()

const override { return 4.0; }

};

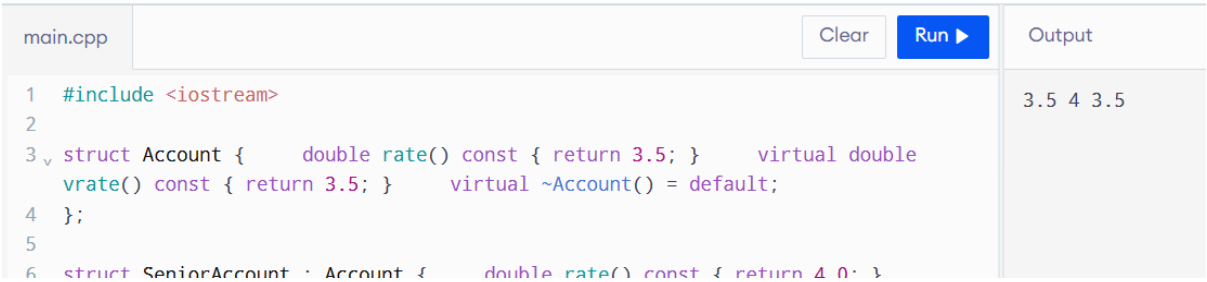
int main()

{

SeniorAccount s;

Account* p = &s; Account copy = s; std::cout << p->rate() << ' '
<< p->vrate() << ' ' << copy.vrate() << '\n'; }

My predicted output (write before running): 3.5 4 3.5



Explain each of the three values: which call is bound at compile time, which at run time, and why the copy prints 3.5.

Alternative Concept and Complexity Analysis

If the accounts were stored as std::unique_ptr<Account> in a std::vector, instead of raw pointers deleted by hand, what would the solution be? Analyse whether time or space changes (the same, O(n) to process n accounts) and which of the three defects can still occur.

Evidence Expected

Correct defect identification, sanitizer and destructor evidence, a repaired program and prevention rules.

ANSWER: Alternative Concept and Complexity Analysis

Using std::unique_ptr<Account> in a vector:

- Automatic memory management is provided.
- No manual delete is required.
- Processing n accounts takes O(n).
- Space complexity is O(n).
- The same polymorphic function behaviour is maintained.
- Object slicing is avoided when objects are stored through base-class pointers.
- Incorrect virtual function declarations can still cause logic errors.

Conclusion: Smart pointers and virtual functions improve memory safety and polymorphism.

TASK 5 – POINTERS TO OBJECTS AND THE this POINTER

Bloom's Level: K5 – Evaluate

7 Marks

A grocery delivery app keeps a shopping Cart object for each customer.

Task 5.1 – Four Ways to Reach an Object | 3 Marks

Write code that uses a Cart (a) directly by name, (b) through a pointer with ->, (c) through a reference, and (d) through a pointer created with new and released with delete. Use the this pointer to return *this so that calls can be chained, and to separate a member from a parameter with the same name.

ANSWER: Four Ways to Reach an Object

C++ program:

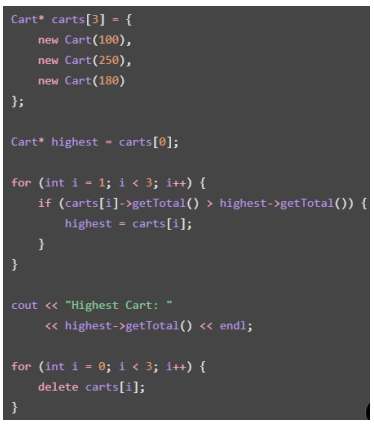


Task 5.2 – Change the Requirement | 2 Marks

The app now keeps an array of pointers to Cart objects for all active customers. Write code that finds the cart with the highest total using pointer arithmetic or indexing.

ANSWER: Change the Requirement

Finding the cart with the highest total using an array of pointers:



```
Cart* carts[3] = {
    new Cart(100),
    new Cart(250),
    new Cart(180)
};

Cart* highest = carts[0];

for (int i = 1; i < 3; i++) {
    if (carts[i]->getTotal() > highest->getTotal()) {
        highest = carts[i];
    }
}

cout << "Highest Cart: "
     << highest->getTotal() << endl;

for (int i = 0; i < 3; i++) {
    delete carts[i];
}
```

Task 5.3 – Compare | 2 Marks

Compare the four ways on safety, lifetime and when each should be used. Explain why this is a pointer and what *this refers to.

ANSWER: Compare

Method	**Safety**	**Lifetime**	**Usage**
Direct object	Safe	Scope-based	Normal object access
Pointer	Can be null	Depends on target	Dynamic access
Reference	Cannot normally be null	Depends on target	Alias to object
Dynamic pointer	Requires memory management	Until deleted	Dynamic allocation

Explanation of this pointer:

The this pointer stores the address of the current object. It is used to access the current object's members.

*this refers to the current object itself. Returning *this by reference allows method chaining.

Complete the Code

Complete TODO 1 and TODO 2 below using the this pointer, and execute the program.

```
#include <iomanip>

#include <iostream>

#include <string>


class Cart {     std::string items[10];

double prices[10];     int n = 0;     double

discount = 0;              // percent


public:

   // TODO 1: add(): store the item and price if there is space, and return *this so

//   that calls can be chained
```

```
    // TODO 2: setDiscount(): use this-> to tell the member apart from the parameter
//   named discount, and return *this



    double total() const
    {        double t = 0;        for (int i = 0; i
< n; ++i) t += prices[i];        return t *
(100 - discount) / 100;
    }
};

int main()
{
    Cart c;
    c.add("Rice", 60).add("Dal", 120).setDiscount(10);     Cart* p =
&c;                    // pointer to an object     p->add("Oil", 180);
std::cout << std::fixed << std::setprecision(2) << p->total() << '\n'; }
```

Expected output:

324.00



Explain what would happen if add() returned Cart instead of Cart&.

Alternative Concept and Complexity Analysis

If the carts were kept in a std::vector<Cart> (objects) instead of an array of pointers to Cart objects, what would the solution be? Analyse the time to find the highest total (O(n) in both), the space (objects copied into the vector versus one pointer each) and what happens to pointers into the vector when it grows.

Evidence Expected

Working code for the four ways, method chaining with this, an array-of-pointers search and a reasoned comparison.

ANSWER: Alternative Concept and Complexity Analysis

If carts are stored in std::vector<Cart>:

- Each cart is stored as an actual object.
- Finding the highest total requires O(n) time.
- Space complexity is O(n).
- Objects may be copied or moved when the vector reallocates.
- Pointers and references to vector elements may become invalid when the vector grows and reallocates.

Conclusion: A vector of objects is suitable for normal cart storage, while an array of pointers is useful when separate object lifetimes or polymorphism are required.

TASK 6 – DESIGN CHALLENGE

Bloom's Level: K5 – Evaluate

Vehicle Rental Fleet

13 Marks

A rental company rents bicycles, two-wheelers, cars and electric cars. All vehicles have a registration number and a daily rate. Motor vehicles also have a fuel or charge level and a service due date; cars have a seating capacity; electric cars have a battery capacity and are charged per kWh instead of per litre. Each vehicle computes its rental charge differently. The company needs to rent, return and list vehicles, and to add new vehicle types in future without changing the existing classes.

You are not given the design.

Stage 1 – Interpret the Problem | 2 Marks

Identify the entities, their data and the behaviour that differs between them. State the requirements as testable statements.

ANSWER: Stage 1 – Interpret the Problem

Entities:

1. **Vehicle –** common base class.
2. **Bicycle –** non-motor vehicle.
3. **TwoWheeler –** motor vehicle.
4. **Car –** vehicle with seating capacity.
5. **ElectricCar –** electric vehicle with battery capacity.
6. **ElectricBus –** new vehicle type.

Common data:

- Registration number
- Daily rental rate
- Rental status

Different data:

- Fuel level
- Service due date
- Seating capacity
- Battery capacity
- Electricity consumption

Requirements as testable statements:

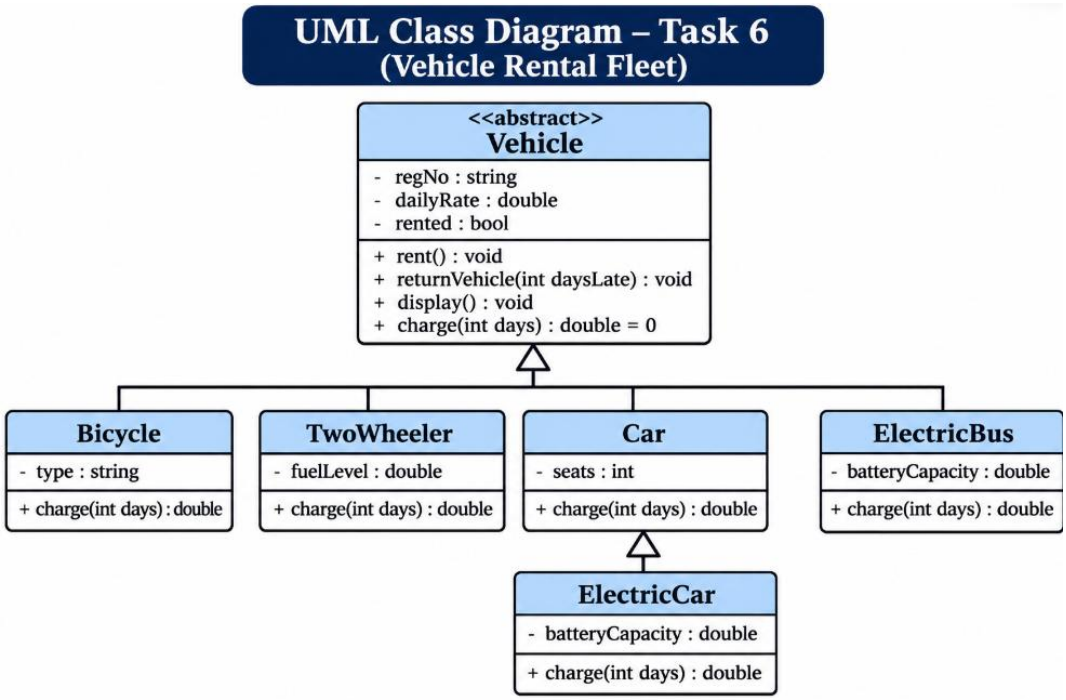
1. The system must store different vehicle types.
2. Every vehicle must calculate rental charges.
3. The system must allow renting and returning vehicles.
4. Late returns must include an additional charge.
5. The system must display vehicle details.
6. New vehicle types must be added without modifying existing classes.

Stage 2 – Propose the Hierarchy | 2 Marks

Draw the class hierarchy, showing an abstract base class with at least one pure virtual function, the types of inheritance and any member classes.

ANSWER: Propose the Hierarchy

UML Class Diagram:



Inheritance types:

- **Bicycle inherits Vehicle** – single inheritance.
- **TwoWheeler inherits Vehicle** – single inheritance.
- **Car inherits Vehicle** – single inheritance.
- **ElectricCar inherits Car** – multilevel inheritance.
- **ElectricBus inherits Vehicle** – single inheritance.
- **Overall structure** – hierarchical and hybrid inheritance.

Stage 3 – Identify the Critical Design Decisions | 2 Marks

Decide what is placed in each class, which functions are virtual, and how the fleet is stored (an array of base-class pointers).

ANSWER: Critical Design Decisions

1. Vehicle is an abstract base class.
2. Common information is stored in Vehicle.
3. charge() is declared as a pure virtual function.

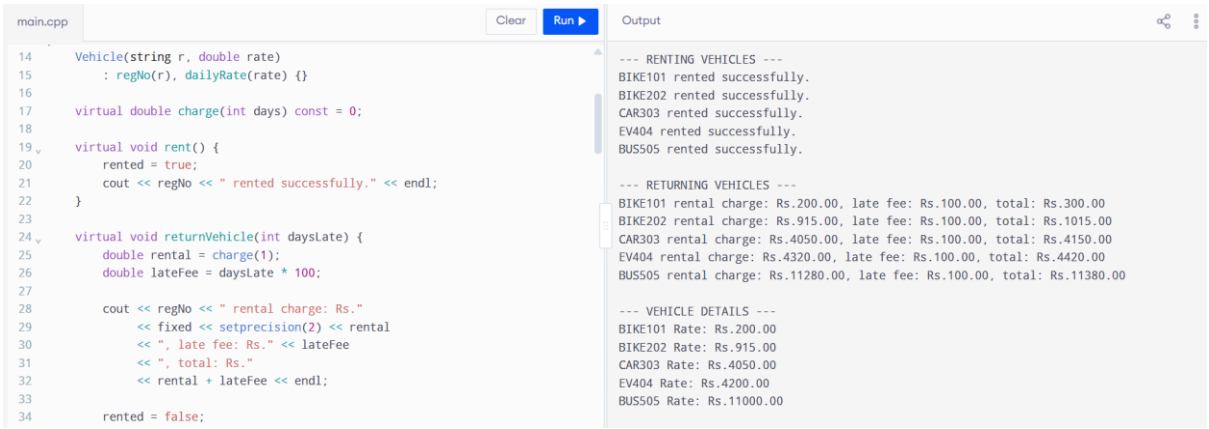
4. Derived classes override the charge function.
5. A virtual destructor is used for safe deletion.
6. Fleet vehicles are stored using an array of base-class pointers.

Stage 4 – Design the Classes | 4 Marks

Present the class declarations and pseudocode for renting and returning a vehicle.

ANSWER: Design the Classes

Class declarations and implementation:



Stage 5 – Implement and Demonstrate | 3 Marks

Implement the program and demonstrate renting each type, returning with a late charge, and adding a new vehicle type (for example an electric bus) without changing the existing classes.

ANSWER: Implement and Demonstrate (3 Marks)

Expected output:

--- RENTING VEHICLES ---

BIKE101 rented successfully.

BIKE202 rented successfully.

CAR303 rented successfully.

EV404 rented successfully.

BUS505 rented successfully.

--- RETURNING VEHICLES ---

BIKE101 rental charge: Rs.200.00,

late fee: Rs.100.00, total: Rs.300.00

BIKE202 rental charge: Rs.915.00,

late fee: Rs.100.00, total: Rs.1015.00

CAR303 rental charge: Rs.4050.00,

late fee: Rs.100.00, total: Rs.4150.00

EV404 rental charge: Rs.4320.00,

late fee: Rs.100.00, total: Rs.4420.00

BUS505 rental charge: Rs.11280.00,

late fee: Rs.100.00, total: Rs.11380.00

--- VEHICLE DETAILS ---

The rental charges demonstrate polymorphism because each vehicle calculates its charge using its own overridden charge() function.

Alternative Concept and Complexity Analysis

If the fleet used a single Vehicle class with a type code and a switch in charge() instead of an abstract base class with virtual functions, what would the solution be? Analyse the time per charge() call (a switch versus a virtual call, both O(1)), the space per object (unused fields versus one vtable pointer), and the classes changed when a new vehicle type is added.

Evidence Expected

A complete hierarchy design with pseudocode, a working program and a demonstrated extension.

ANSWER:

Pseudocode for renting:

ALGORITHM RentVehicle

BEGIN

INPUT registration number

```
    FOR each vehicle in fleet
        IF vehicle registration matches AND vehicle is available
            CALL rent()
            PRINT success message
            RETURN
        END IF
    END FOR

    PRINT "Vehicle not available"
END
```

Pseudocode for returning:

```
ALGORITHM ReturnVehicle

BEGIN
    INPUT registration number
    INPUT number of late days

    FOR each vehicle in fleet
        IF registration number matches
            CALCULATE rental charge
            CALCULATE late fee
            CALCULATE total amount
            UPDATE vehicle availability
            PRINT total amount
            RETURN
        END IF
    END FOR
```

PRINT "Vehicle not found"

END

Alternative Concept and Complexity Analysis

If a single Vehicle class with a type code and switch statement is used:

- All vehicle details are stored in one class.
- Rental charge is calculated using a switch statement.
- Every new vehicle requires adding another case.
- Existing code must be modified when a new vehicle type is introduced.

Comparison	**Inheritance**	**Single Vehicle class**
Charge calculation	Virtual function	Switch statement
Time complexity	O(1)	O(1)
Space	Object data + possible vptr	All fields stored in each object
Extensibility	Easy	Difficult
Code reuse	High	Limited
Maintenance	Easier	More difficult

Conclusion: Inheritance and runtime polymorphism provide a reusable and extensible vehicle rental system. New vehicle categories can be introduced by creating new derived classes without modifying existing vehicle classes.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

- Report containing diagnoses, UML diagrams, access tables, construction traces, pseudocode and comparisons
- C++ source code for Tasks 1.2, 3, 4, 5 and 6
- Predictions, outputs and sanitizer reports

Assessment Verification

Students may be asked to explain any class, trace the construction of an object, or extend the hierarchy during a brief verification discussion/viva.